REPORT ON

PALMISTRY AND TAROT INTELLIGENCE PLATFORM

1. ABSTRACT

The Palmistry & Tarot Intelligence Platform is a web-based application that brings palmistry and tarot-reading experiences together with personalized insights, guidance, recommendations, reading history, consultations, notifications, and role-specific dashboards. Its architecture separates the browser interface, backend application logic, and persistent database.

The frontend uses React and React Router, with a beige-and-brown luxury mystical theme. The backend uses Python and FastAPI to expose API endpoints. SQLAlchemy provides database access, and PostgreSQL stores persistent application records. The platform supports User, Admin, Tarot Reader, and Spiritual Consultant roles. Supporting feature areas include authentication, profiles, readings, intelligence, consultations, guidance, notifications, and platform feedback.

Docker Compose runs the frontend, backend, and PostgreSQL as separate local services. During deployment troubleshooting, the CORS configuration was updated so the browser frontend could communicate with the backend. Existing data was migrated to a PostgreSQL 18 container, and key record counts were verified. The user subsequently confirmed the local Docker application was working. Public internet hosting is a separate future step.

The project demonstrates full-stack development, REST API integration, relational persistence, authentication, role-aware routing, containerization, migration, debugging, and deployment preparation. Palmistry and tarot interpretations should be understood as reflective or entertainment-oriented content, not guaranteed predictions or professional advice.

2. INTRODUCTION

Palmistry and tarot experiences are often provided through separate tools or manual consultations. A single digital platform can provide a consistent interface for reading inputs, interpretations, history, and related guidance. This project combines those feature areas in a role-aware web application.

3. PROBLEM STATEMENT

The project addresses the need for a centralized application where users can access palmistry and tarot features, revisit earlier readings, receive relevant guidance, and use features according to their role. Separate dashboards organize administrative, tarot-reader, and spiritual-consultant activities.

4. OBJECTIVES

1) Combine palmistry and tarot experiences in one web application.
2) Provide registration, login, authentication, and role-specific navigation.
3) Support reading features, insights, guidance, recommendations, and reading history.
4) Provide dedicated dashboards for User, Admin, Tarot Reader, and Spiritual Consultant.
5) Persist application data in PostgreSQL through SQLAlchemy.
6) Expose backend functionality through FastAPI endpoints.
7) Keep frontend pages, backend routers, and data models maintainable.
8) Support in-app notifications and engagement features.
9) Run services consistently through Docker Compose.
10) Prepare for a later secure public deployment.

5. SCOPE

The scope includes React frontend pages, FastAPI backend APIs, PostgreSQL persistence, authentication and roles, palmistry and tarot feature areas, profile and history functions, consultation and guidance routes, notifications, platform feedback, and local Docker deployment. Feature completeness must be confirmed against the final source code and tests.

6. INTENDED USERS

Role	Purpose
User	Access personal dashboard and reading-related features.
Admin	Access administration and platform oversight functions.
Tarot Reader	Use the dedicated tarot-reader dashboard and workspace.
Spiritual Consultant	Use consultant and consultation/guidance-related workflows.

7. SYSTEM REQUIREMENTS

TECHNOLOGIES

Layer	Technology	Purpose
Frontend	React	Interactive browser interface.
Routing	React Router	URL-to-page mapping and role-aware route guards.
Icons	lucide-react	Navigation and interface icons.
Backend	Python, FastAPI	API endpoints, validation, and server-side logic.
ORM/data access	SQLAlchemy	Model mapping and database interaction.
Database	PostgreSQL	Persistent users, readings, consultations, and related data.
Application server	Uvicorn	Runs the FastAPI application.
Containerization	Docker	Packages services into containers.
Orchestration	Docker Compose	Runs and connects application services.
Development environment	Windows	Current local development environment.

ENVIRONMENT

A Windows computer capable of running Docker Desktop, Docker Compose, and a modern browser is used. Exact minimum hardware requirements and installed Python, Node.js, and Docker versions should be copied from the actual environment rather than estimated.

8. ARCHITECTURE AND WORKFLOW

THREE – TIER ARCHITECTURE

Layer	Responsibilities	Communication
Presentation	React pages, forms, navigation, dashboards, visual output.	Browser makes HTTP requests to APIs.
Application	FastAPI routes, validation, authentication, role checks, business logic.	Calls database layer and returns JSON/errors.
Data	PostgreSQL tables and persistent records.	Accessed through SQLAlchemy.

REQUEST LIFECYCLE

1. User opens the React application in a browser.
2. User signs in or opens a permitted feature.
3. Frontend sends an HTTP request to the relevant FastAPI endpoint.
4. Backend validates input and checks authentication/permissions where required.
5. SQLAlchemy reads or writes PostgreSQL records.
6. Backend returns JSON or an appropriate HTTP error.
7. Frontend updates the page or displays a message.

FRONTEND

React implements the interface and shared layout/navigation. React Router maps URLs to pages and supports protected, role-aware routes. An authentication context manages login state and logout. The established beige-and-brown luxury mystical theme is a project constraint; deployment configuration should not require an unrelated redesign.

BACKEND

The FastAPI application is assembled in backend/app/main.py and registers routers for authentication, profiles, readings, intelligence, administration, tarot readers, consultants, notifications, consultations, guidance, and platform feedback. A health endpoint is used to check backend availability.

DATABASE LAYER

SQLAlchemy models represent persistent entities and the database engine is configured in the backend database module. Startup has used Base.metadata.create_all(bind=engine) within an application lifespan. This can create missing mapped tables, but a production system should use controlled, versioned schema migrations.

AUTHENTICATION and AUTHORIZATION

The login endpoint uses OAuth2PasswordRequestForm. The observed flow normalizes the email supplied through the form username field, retrieves the account, verifies the submitted password against its stored hash, and rejects inactive accounts. The frontend stores the bearer token under localStorage key palmistry_token and uses authentication state for protected pages. Role-aware redirects send users to their relevant dashboards.

SECURITY CONSIDERATION

A public deployment needs HTTPS, strong production secrets, exact CORS origins, protected environment variables, and reviewed authorization. Never commit production credentials or database URLs to a public repository.

9. MODULES and ROLE-BASED ACCESS

AUTHENTICATION

Authentication covers login, registration, current-user retrieval, token handling, logout, and role-aware navigation. Referenced endpoints include POST /api/auth/login and /api/auth/me. Google sign-in was discussed; production OAuth configuration and callback behavior must be verified separately.

USER DASHBOARD and NAVIGATION

The regular-user navigation has included Dashboard, Palm Reading, Tarot Reading, Life Trends, Insights, Guidance, Recommendations, History, and Intelligence.

The dashboard provides entry to user-facing features, while profile/history pages support personalization and revisiting activity.

PALM READING

1) Capture palm images using a live camera.
2) Upload palm images from a device.
3) Display the captured or uploaded palm image.
4) Analyze relevant palm features using computer vision.
5) Display palm-line overlays on the palm image.
6) Present supported palm-reading results.
7) Save reading data where implemented.
8) Verify the computer-vision library, model, and line-detection method in the source code.

TAROT READING

1) Include all 78 Tarot cards:
    - 22 Major Arcana cards.
    - 56 Minor Arcana cards.
2) Allow users to choose a Tarot spread.
3) Reveal selected cards.
4) Display Tarot card images.
5) Provide interpretations and insights.
6) Generate recommendations based on the current reading.
7) Expand interpretation text to provide more detailed explanations.

8) Verify the card-selection algorithm, available spreads, and AI integration in the source code.

TRENDS, INSIGHTS, GUIDANCE and RECOMMENDATIONS

1) Organize reading information into summaries.
2) Present insights based on reading context.
3) Provide guidance and suggested next steps.
4) Display personalized recommendations.
5) Update recommendations according to different palm and Tarot readings.
6) Support guidance-effectiveness tracking in the consultant workflow.
7) Verify metric definitions and update triggers in the source code.

READING HISTORY

1) Display previously stored reading activities.
2) Allow users to revisit past readings.
3) Support access to historical reading information.
4) Use stored reading data where implemented.
5) A total of 94 reading records was counted during database migration.
6) Note: The count may differ in another deployment or database.

ADMIN

1) Provide a dedicated Admin dashboard.
2) Restrict access through role-based navigation and route protection.
3) Support platform oversight.
4) Provide management and analytics features where implemented.
5) Verify the available management actions against the current source code and screenshots.

TAROT READER

1) Provide a dedicated Tarot Reader dashboard.
2) Include a Tarot Workspace.
3) Support tarot-reader activities and related records.
4) Provide report export functionality.
5) The report export endpoint returned HTTP 200 during development.
6) Verify the workspace actions, timing behavior, and report fields in the final build.

SPIRITUAL CONSULTANT

1) Provide a dedicated Spiritual Consultant dashboard.
2) Display consultant profile information.
3) Display consultant dashboard data.
4) Support consultation-related workflows.
5) Organize client trends and outcomes.
6) Track guidance effectiveness where implemented.
7) Support feedback-related workflows.
8) Test with a consultant account to confirm that actions update the correct metrics.

CONSULTATIONS

1) Manage consultation-related records.
2) Connect consultation workflows with the consultant role.
3) Support consultation data operations.
4) A backend HTTP 500 error caused by an invalid regular expression was encountered during development.
5) The pattern was corrected in backend/app/routers/consultations.py.
6) Verify the consultation lifecycle, fields, and current implementation.

NOTIFICATIONS and ENGAGEMENT

1) Provide daily guidance notifications.
2) Send reading reminders within the in-app notification system.
3) Provide insight updates.
4) Provide spiritual-growth alerts.
5) Display platform announcements.
6) List user notifications.
7) Display unread notification counts.
8) Mark individual notifications as read.
9) Mark all notifications as read.
10) Delete notifications.
11) Manage notification preferences.
12) Allow administrators to publish announcements.
13) Use database-backed notifications where implemented.
14) Avoid duplicate daily, weekly, or monthly notifications where supported.

PLATFORM FEEDBACK

1) Provide a platform feedback workflow.
2) Support feedback related to platform experiences.
3) Support guidance-related feedback where implemented.
4) Connect feedback with relevant workflows.
5) Verify the feedback fields, access permissions, and metric calculations in the source code.

10. FUNCTIONAL WORKFLOWS

LOGIN WORKFLOW

1) User enters email and password on the login page.
2) Frontend sends credentials using the expected form encoding.
3) Backend normalizes email, retrieves account, verifies password hash, and checks account status.
4) On success, backend returns a bearer token.
5) Frontend stores token and loads authenticated user details.
6) Role-aware routing opens the appropriate dashboard.

PALM READING WORKFLOW

1) User opens Palm Reading.
2) User captures or selects an image if that input method is supported in the current build.
3) Frontend displays the image and submits it to the configured analysis endpoint.
4) Analysis layer processes the input and returns supported findings.
5) Frontend presents image, any supported overlays, and interpretation.
6) Result is stored in history if persistence is enabled for the flow.

TAROT READING WORKFLOW

1) User opens Tarot Reading and chooses a spread.
2) Application selects/reveals the required number of cards from the deck.
3) Card images and spread positions are displayed.
4) Application presents card meanings and combined interpretation.
5) Reading is saved to history where implemented; related insights/recommendations may refresh.

CONSULTATION and GUIDANCE

1) The consultant opens the Consultant Dashboard.
2) The consultant accesses consultation and guidance functions.
3) The backend processes consultation and guidance requests.
4) SQLAlchemy manages related database records.
5) Feedback data is stored where implemented.
6) Action-completion data is recorded where implemented.
7) Outcome summaries are updated based on available records.
8) Guidance-effectiveness summaries are updated where supported.
9) Functional tests verify that dashboard metrics reflect the latest records.

NOTIFICATION WORKFLOW

1) The frontend requests notification records from the backend.
2) The frontend retrieves the unread notification count.
3) The backend fetches stored notifications from the database.
4) The backend may generate eligible notifications based on activity and engagement rules.
5) The notification center displays available notifications.
6) Users can mark notifications as read.
7) Users can manage notification preferences.
8) Users can view unread notification counts.
9) Users can delete notifications where supported.
10) Administrators can publish platform announcements.
11) Admin announcement requests require authorization.
12) Functional tests verify notification generation, read status, preferences, and access permissions.

TESTING AND VALIDATION

1) Login and registration validation.
2) Role-based access checks for all four roles.
3) Palm image upload and camera functionality.
4) Tarot card selection, spreads, and interpretation display.
5) Reading history and recommendation updates.
6) Consultation feedback and guidance-effectiveness metrics.
7) Notification creation, read status, and preferences.
8) API response checks and error handling.
9) Docker frontend, backend, and database connectivity.

11. DATABASE DESIGN AND API OVERVEW

DATABASE

PostgreSQL is the persistent database in the current Docker environment. SQLAlchemy provides model mapping and database access. Startup creates missing mapped tables, while production schema changes should use controlled migrations and backups.

Entity group	High-level purpose
Users/accounts	Identity, role, active state, password hash.
Profiles	Profile, preference, and context data where modelled.
Readings	Stored palm/tarot reading activity and results.
Consultations	Consultation records.
Notifications	In-app notification records.
Notification preferences	User notification preferences.
Guidance/feedback	Guidance and feedback-related data where defined.

This is a conceptual list, not a definitive schema. Exact table names, columns, keys, indexes, constraints, and relationships should be taken from current SQLAlchemy models and used to prepare the final ER diagram.

DOCKER

Docker is used to containerize and run the Palmistry and Tarot Intelligence Platform. Docker Compose manages the frontend, FastAPI backend, and PostgreSQL database as separate services. It simplifies application setup, service communication, and local deployment while providing a consistent development environment.

CONCLUSION

The Palmistry and Tarot Intelligence Platform integrates palmistry, tarot reading, personalized guidance, recommendations, and consultation management into a single web application. It uses React, FastAPI, SQLAlchemy, PostgreSQL, and Docker to provide role-based access and data management. The project demonstrates full-stack development and provides a foundation for future improvements, testing, and public deployment.